动态规划

简介

- 动态规划简称DP，是在20世纪50年代由一位卓越的美国数学家Richcard Bellman发明的。它作为一种重要的工具在应用数学中被广泛的应用。它不仅可以解决特定类型的优化问题，还可以作为一种通用的算法设计技术来使用。

DP的实质

- 利用问题的所具有的重叠子问题的性质进行记忆化求解。（用空间换时间）

- 求Fibonacci数：

$$f(n) = f(n-1) + f(n-2) \qquad n>2$$

$$f(1)=f(2)=1$$

常规递归

```
int Non_DP(int n)
  {
    if (n==1 || n==2)
      return 1;
    else
      return Non_DP(n-1) + Non_DP(n-2);
  }
```

- 指数级时间复杂度，无法忍受

两种实现方式

- 自底向上(bottom up)

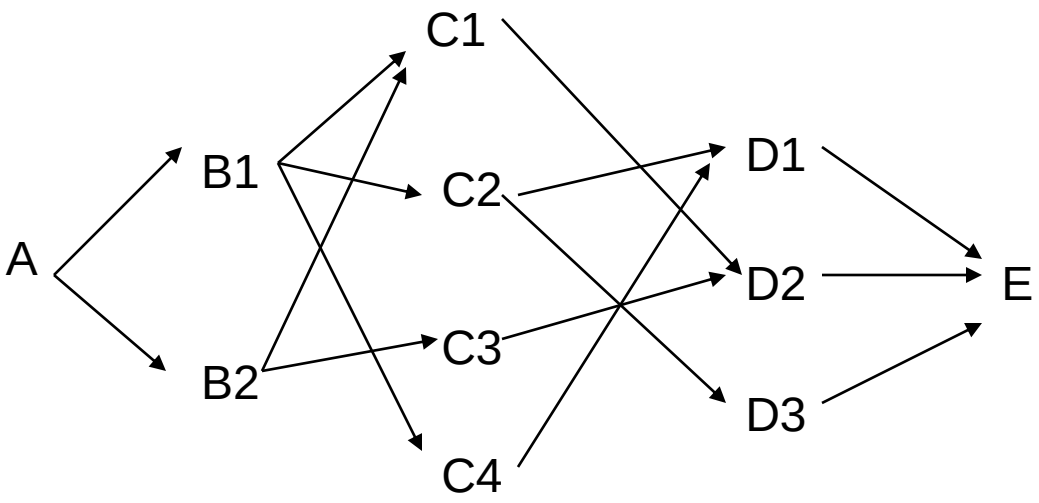
```
int DP_Bottom_Up(int n)
{
    memo[1] = memo[2] = 1;
    for (int i=3; i<=n; i++)
        memo[i] = memo[i-1] + memo[i-2];
    return memo[n];
}
```

- 自顶向下（top down）　（这样写法也叫记忆搜索）

```
int DP_Top_Down(int n)
{
    if (n == 1 || n == 2)   return 1;
    if (memo[n] != 0)    return memo[n];
    memo[n] = DP_Top_Down(n-1) +
            DP_Top_Down(n-2);
    return memo[n];
}
```

基本概念

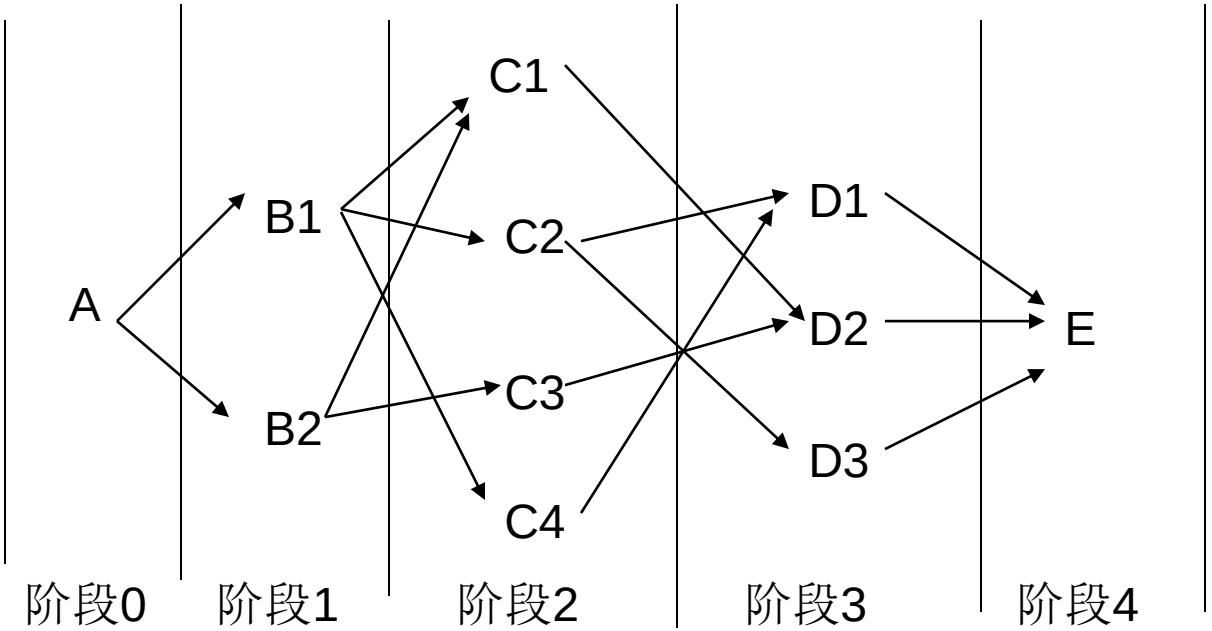
- 最短路问题



求A到E的最短路径。

- 直观的方法是用回溯法搜索。时间复杂度为指数级。
- 低效的原因：没有充分利用重叠子问题的性质。

- 此图有明显的次序，可以划分为5阶段。



- 设 $Dis^{[k]}[x]$ 为第k阶段城市x到城市E的最短路径长度。map[i][j]为i，j两个城市间的距离。

- 递归方程为
  $Dis^{[k]}[x] = \min \{ Dis^{[k+1]}[y]+map[x,y] \}$

- 此问题时间复杂度降为$O(n^2)$.

- 状态：贴切，简洁的描述出事物性质的单元量。例如：Dis[x]。

  要求：状态与状态之间可以转移，以便有初始状态逐渐转移到目标状态，找到问题的解。

- 阶段：若干性质相近可以同时处理的状态的集合。就是计算状态的顺序。

  要求：每个阶段中状态的取值**只与这个阶段之前的阶段中的状态有关**，与这个阶段之后的阶段中的状态无关。

- 状态转移方程：前一个阶段中的状态转移到后一个阶段的状态得演变规律，即相邻两个阶段的状态变化方程。

$$f_k(i) = opt\ \{\ f_{k-1}(j) + cost(i,j)\ \}$$

k阶段的i状态与k-1阶段的j状态有关

- 决策：计算每个状态时作出的选择。

- 无后效性：决策之取决于当前状态的特征因素，而和到达此状态的方式无关。也就是每个阶段中状态的取值只与这个阶段之前的阶段中的状态有关，与这个阶段之后的阶段中的状态无关。

- 如果当前定义的状态不满足无后效性，应重新定义。

一维状态存储问题

- 硬币问题1：

  有n种硬币，每种硬币的面值为v_i元，且只有一枚，问用这n种硬币找零S元的方法数。

- 设d[i][j]为前i种硬币组成j元的方法数。

  $d[i][j] = d[i-1][j] + d[i-1][j-v_i]$

  $d[0][0]=1, d[0][1 \ldots S]=0$

- 空间复杂度为$O(n^2)$,浪费！

```
d[0]=1; d[1…S]=0;
for (i=1; i<=n; i++)
{
    for (j=S; j>=vi; j--)
    {
        d[j] += d[j-vi];
    }
}
```

- 0-1背包问题：

  给定n种物品和一背包。物品i的重量是wi，其价值为vi，背包的容量为c。问应如何选择装入背包中的物品，使得装入背包中物品的总价值最大?

- 设m(i,j)是背包容量为j，可选择物品为i，i+1，…,n时，0-1背包问题的最优值。
- $m(i,j) = \max\{ m(i+1,j), m(i+1,j-wi)+vi\}$ $j>=wi$

  $m(i+1,j)$ $0<=j<wi$

  $m(n,j) = vn$ $j>=wn$

  0 $0<=j<wn$

```
d[0…c]=0;
for (i=1; i<=n; i++)
{
    for (j=c; j>=wi; j--)
    {
        d[j] = max(d[j],d[j-wi]+vi);
    }
}
```

- 推荐题目：
  http://poj.org/problem?id=1384 Piggy-Bank

- 硬币问题2：

  有n种硬币，每种硬币的面值为v_i元，有m_i枚，问用这n种硬币找零S元的方法数。

```
d[0]=1;d[1…S]=0;
for (i=1; i<=n; i++)
{
	for (j=S; j>=vi; j--)
	{
		for(k=1;k<=mi;k++)
		{
			if (j-k*vi>=0) d[j] += d[j-k*vi];
		}
	}
}
```

- 硬币问题3：

  有n种硬币，每种硬币的面值为v_i元，有无数枚，问用这n种硬币找零S元的方法数。

```
d[0]=1;d[1…S]=0;
for (i=1; i<=n; i++)
{
	for (j=S; j>=vi; j--)
	{
		for(k=1;;k++)
		{
			if (j-k*vi>=0) d[j] += d[j-k*vi];
			else break;
		}
	}
}
```

```
d[0]=1; d[1…S]=0;
for (i=1; i<=n; i++)
{
    for (j=vi;j<=S; j++) //无数的时候是顺向
    {
        d[j] += d[j-vi];
    }
}
```

- 推荐题目：

  http://acm.hdu.edu.cn/showproblem.php?pid=1248 寒冰王座

  http://acm.hdu.edu.cn/showproblem.php?pid=1171 Big Event in HDU

- http://poj.org/problem?id=2614 Old Wine Into New Bottles

  有n种小瓶子，每种瓶子容量范围是V_{i1}——V_{i2}，要灌满容量为L的大瓶子。问最少差多少体积没有灌满。

- 设d[i]表示体积为i是否可以达到。若v可取到，则

  d[i]=d[i] || d[i-v]

- d[0]=1;d[1…L]=0;
- for(i=0;i<n;i++)

```
{
    for(j=0;j<=L-v_{i1};j++)
    {
        if (d[j]==1)
            for(k=v_{i1};k<=v_{i2};k++)
            d[j+k]=1;
    }
}
```

- 最大子段和问题：

  给定由n个整数(可能为负整数)组成的序列a1,a2,…,an，求该序列形如的 $\sum_{k=i}^{j} a[k]$ 子段和的最大值。

- 设b_i为到a_i截至且包括a_i的字段最大和。

```
int maxsum()
{
        int sum=0,b=0;
        for(int i=0;i<n;i++)
        {
                if (b>0) b+=a[i]; //正的就连接起来
                else b=a[i];    //不然重新来过
                if (b>sum) sum=b;    //sum记录的一直是最大的值
        }
        return sum;
}
```

- http://acm.hdu.edu.cn/showproblem.php?pid=1231 **最大连续子序列**
- 引申问题：

  最大子矩阵和问题

  最大m段子段和问题

- 最长递增子序列（LIS）
- 常规DP，时间复杂度为$O(n^2)$。
- 存在一种特殊的方法，时间复杂度为O（n*logk)。

- 问题描述：给定N（N <= 1000000）个数，请在其中找出最长的一个上升链（严格上升）。
- 例：1　10　2　9　3　8　4　7　5　6
- 其中最长上升序列长度为6，是：1 2 3 4 5 6。

- 算法1：
- 我们令f[i]表示前i个数中找到的以第i个数num[i]结尾的最长链的长度。那么，很容易列出方程
- f[0] = 0;
- f[i] = max(f[j] | j < I and num[j] < num[i]) + 1 1 <= I <= N
- 最后结果是：ans = max(f[i] | 1 <= I <= N)

- 上述算法的复杂度是O(N^2)，类似第一题，引入s[i]表示f[i]是由哪个状态转移过来的，则可逆推出最长升链。
- 现在我们再看一下这个问题的一个变形：请你用最少的不上升序列覆盖这N个数。

- 上面的例子最少要用6条非升链覆盖：
- 1
- 2
- 3
- 4
- 5
- 10　9　8　7　6
- 再举一个例子(N = 11)：100 89 99 89 76 75 92 83 54 90 1

- 要用3条非升链：
- 100 89 89 76 75 54 1
- 99 92 83
- 90
- 而我们再观察一下它的最长升链，其中一条是：75 83 90，长度也为3！

- 这是巧合还是必然？
- 这是必然！！！可以证明非升链覆盖问题和最长升链问题是同解的。同样，升链覆盖、降链覆盖、非降链覆盖和最长非升链、最长非降链、最长降链同解。
- 口诀：前面加“非”字~
- 大家可以想象证明，等大家了解了算法2，就自然明白了。

- 算法2：
- 由于N的规模，算法一是在太慢了。这里介绍一种O(nlogn)的算法。
- 首先定义数组D：D[i]（1 <= I <= N）表示目前为止，所有长度为i的升链中，最后一个元素的最小可能值。如当前找到两个长度为3的升链：2 3 4；1 3 9 那么D[3] = 4。显然，对于同样长度的升链，结尾越小越容易拓展成更长的升链。并且可知D是一个严格升链（若其中D[i+1] <= D[i]，则去i+1链中第i个元素赋给D[i]）

- 我们还设置一个变量s表示目前为止找到最长链的长度。初始s=0, D赋正无穷。
- 然后依次扫描数列中的每一个数。譬如现在扫到第i个数，首先在D中二分查找num[i]可以放置的位置p，满足D[p] >= num[i] and (p = 0 or D[p – 1] < num[i])。将num[i]写入D[p]，如果p>s，说明找到更长的链，更新s。

- 例如：1 3 2 5 4 5
- initial s = 0
- step1 s = 1 D[1] = 1
- step2 s = 2 D[1] = 1 D[2] = 3
- step3 s = 2 D[1] = 1 D[2] = 2
- step4 s = 3 D[1] = 1 D[2] = 2 D[3] = 5
- step5 s = 3 D[1] = 1 D[2] = 2 D[3] = 4
- step6 s = 4 D[1] = 1 D[2] = 2 D[3] = 4 D[4] = 5

- 显然这个算法是O(nlogn)的。
- 同样这个算法证明了前面讨论的变形的正确性。首先，本算法实际上构造出了用最长升链长度（MAXL）条非升链覆盖这N个数的一组解。我们只需要证明至少要用MAXL条非升链才能完成覆盖即可，显然，最长升链就需要MAXL条降链覆盖。证毕。

- 推荐题目：

  http://poj.org/problem?id=1631 Bridging Signals

  http://poj.org/problem?id=2336 Ferry Loading II

二维状态存储问题

- 矩阵连乘问题 （MCM）
- A(m*n) * B(n*q) cost=m*n*q
- A1 (10 x 100), A2 (100 x 5), A3 (5 x 50)

  (A1 (A2 A3)) = 100 x 5 x 50 + 10 * 100 * 50 = 75000

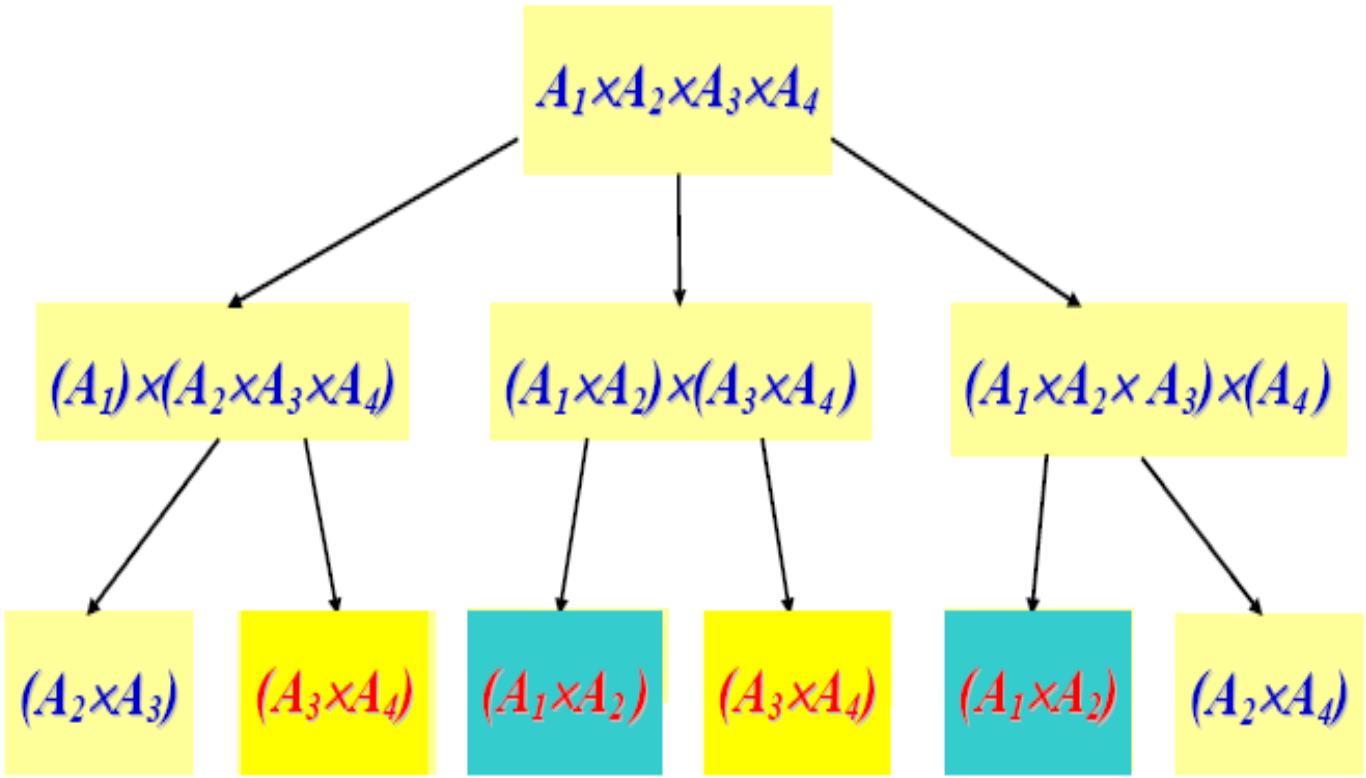
  ((A1 A2) A3) = 10 x 100 x 5 + 10 x 5 x 50 = 7500

- 最优子结构：

$$A_{i\text{-}j}=A_i \times A_{i+1} \times \ldots \times A_j$$

$$A_{1\sim n}=A_{1\sim k}\times A_{k+1\sim n}$$

- 重叠子问题：


$A_1 \times A_2 \times A_3 \times A_4$
$(A_1) \times (A_2 \times A_3 \times A_4)$
$(A_1 \times A_2) \times (A_3 \times A_4)$
$(A_1 \times A_2 \times A_3) \times (A_4)$
$(A_2 \times A_3)$
$(A_3 \times A_4)$
$(A_1 \times A_2)$
$(A_3 \times A_4)$
$(A_1 \times A_2)$
$(A_2 \times A_4)$

$$m[i, j] = 0 \qquad \text{if} \quad i=j$$

$$m[i, j] = \min_{i \le k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} \qquad \text{if} \quad i<j$$

- 阶段划分：

m[1,1]	*m[1,2]*	*m[1,3]*	*m[1,4]*	*m[1,5]*
	m[2,2]	*m[2,3]*	*m[2,4]*	*m[2,5]*
		m[3,3]	*m[3,4]*	*m[3,5]*
			m[4,4]	*m[4,5]*
				m[5,5]